



A. MỤC TIÊU:

- Sử dụng được các widget cơ bản của flutter như: Scaffold, AppBar, Drawer, bottomNavigationBar, Column widget, Row widget, Container widget, stack widget.
- Kết hợp được các widget để xây dựng được giao diện hoàn chỉnh

B. DỤNG CỤ - THIẾT BỊ THỰC HÀNH CHO MỘT SV:

STT	Chủng loại – Quy cách vật tư	Số lượng	Đơn vị	Ghi chú
1	Computer	1	1	

C. NỘI DUNG THỰC HÀNH

1. Tóm tắt lý thuyết

Layout widget là một loại widget trong Flutter dùng để sắp xếp và điều chỉnh giao diện người dùng. Nó quyết định cách các widget con được đặt trong giao diện. Có nhiều loại layout widget khác nhau trong Flutter, bao gồm:

a. Scaffold Widget

- Đặc điểm:

- **Scaffold** là một widget cốt lõi trong cung cấp khung layout chuẩn cho ứng dụng theo phong cách Material Design.
- **Scaffold** giống như "bộ xương" của hình, giúp sắp xếp các thành phần UI biến như app bar, drawer, bottom navigation, floating button... một cách và nhất quán.

- Các thuộc tính:

- **AppBar**: Hiển thị thanh tiêu đề (app phía trên cùng màn hình).
- **body**: Phần nội dung chính của màn hình (hiển thị giữa appBar và bottomNavigationBar/floatingActionButton).
- **floatingActionButton (FAB)**: Nút nổi (thường dùng cho hành động chính của màn hình)
- **drawer**: menu bên trái của ứng dụng (có thể kéo ra từ cạnh bên trái)
- **bottomNavigationBar**: thanh điều hướng ở phía dưới của ứng dụng
- **backgroundColor**: Màu nền của ứng dụng

- Lưu ý:

- Mỗi Scaffold thường đại diện cho một màn hình riêng.
- Không lồng nhiều Scaffold vào nhau.



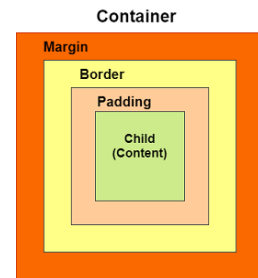
Flutter, dụng
màn
phở
tự động
bar) ở
hình

- Scaffold tự động xử lý Safe Area để tránh phần notch (trên iPhone) hoặc thanh điều hướng (trên Android).

b. Container widget

- Đặc điểm

- **Container** trong Flutter là một widget mẹ có thể chứa widget con và quản lý chúng một cách hiệu quả.
- **Container** phải được đặt trong một widget cha Scaffold widget, Center widget, Padding widget, Row or Column widget làm widget cha của nó.



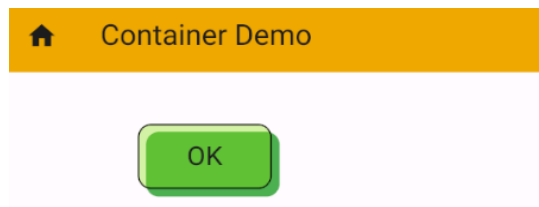
nhiều
như:
widget,

- Một số thuộc tính

- **alignment**: Xác định vị trí của widget con bên trong Container.
- **padding**: Đặt khoảng cách bên trong giữa cạnh của Container và widget con.
- **margin**: Đặt khoảng cách bên ngoài giữa Container và các widget xung quanh.
- **color**: Đặt màu nền cho Container.
- **decoration**: Sử dụng để thêm hiệu ứng hình ảnh như viền, bóng, hoặc hình nền.
- **width và height**: Đặt chiều rộng và chiều cao của Container.
- **constraints**: Xác định các giới hạn về kích thước cho Container.
- **transform**: Áp dụng các phép biến đổi (như xoay, phóng to, thu nhỏ) cho Container.
- **child**: Chứa widget con bên trong Container.

- Ví dụ:

```
Container(
  width: 100,
  height: 50,
  alignment: Alignment.center,
  decoration: BoxDecoration(
    color: Color.fromARGB(90, 134, 227, 4),
    border: Border.all(
      color: const Color.fromARGB(255, 0, 0, 0),
      width: 1,
    ),
    borderRadius: BorderRadius.circular(10),
    boxShadow: const [
      BoxShadow(
        color: Colors.green,
        offset: Offset(6.0, 6.0),
      )
    ],
  ),
  margin: const EdgeInsets.only(left: 100, top: 40),
  child: const Text("OK", style: TextStyle(fontSize: 20))
)
```



c. Row widget và Column widget :

- **Đặc điểm**
 - **Row widget:** dùng để sắp xếp các widget con theo chiều ngang
 - **Column widget:** dùng để sắp xếp các widget con theo chiều dọc
- **Một số thuộc tính**
 - **mainAxisAlignment:** Xác định cách sắp xếp các widget con theo trục chính
 - **crossAxisAlignment:** Xác định cách sắp xếp các widget con theo trục phụ
 - **children:** Chứa các widget con



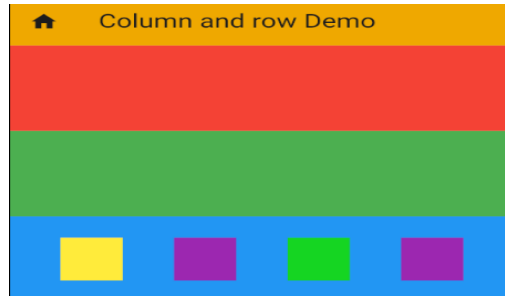
Ví dụ :

```
Column(
  mainAxisAlignment: MainAxisAlignment.start,
  mainAxisSize: MainAxisSize.max,
  crossAxisAlignment: CrossAxisAlignment.stretch,
  children: <Widget>[
    Container(
      color: Colors.red,
      width: 100,height: 100,
    ),
    Container(
      color: Colors.green,
      width: 100,height: 100,
    ),
    Container(
      color: Colors.blue,
      width: 200,height: 100,
      child: Row(
        mainAxisAlignment: MainAxisAlignment.spaceEvenly,
        children: <Widget>[
          Container(
            color: Colors.yellow,
            width: 50,height: 50,
          ),
          Container(
```

```

        color: Colors.purple,
        width: 50,height: 50,
      ),
      Container(
        color: const Color.fromARGB(255, 19, 213, 32),
        ),],)),],))

```



d. Stack widget

- **Đặc điểm:** **stack widget** là widget cho phép chồng nhiều widget lên nhau, các widget con có thể được đặt chồng lấp lên nhau thay vì sắp xếp theo chiều dọc hoặc chiều ngang. Điều này rất hữu ích khi bạn muốn tạo các giao diện có hình nền, lớp phủ, hoặc các phần tử chồng lấp. Thường có 2 loại là stack và IndexedStack
- **Một số thuộc tính**
 - **alignment:** Căn chỉnh các widget con **không được định vị** (non-positioned children) trong Stack.
 - **textDirection:** Xác định hướng văn bản của các widget con.
 - **fit:** Điều chỉnh kích thước của các widget con không được định vị.
 - StackFit.loose: Cho phép widget con có kích thước tự do, nhưng không vượt quá kích thước Stack.
 - StackFit.expand: Ép widget con mở rộng bằng kích thước Stack.
 - **clipBehavior:** **Xác định cách xử lý nội dung vượt ra khỏi biên của Stack.**
 - Clip.none: Không cắt bỏ phần thừa.
 - Clip.antiAlias: Cắt mượt (chậm hơn).
 - **children:** Danh sách các widget con. Có 2 loại widget con:
 - Widget con không định vị: Căn chỉnh theo thuộc tính alignment của Stack
 - Widget con định vị: là các widget con sử dụng widget Positioned để xác định vị trí bằng top, bottom, left, right, v.v.
- **Ví dụ:**

```

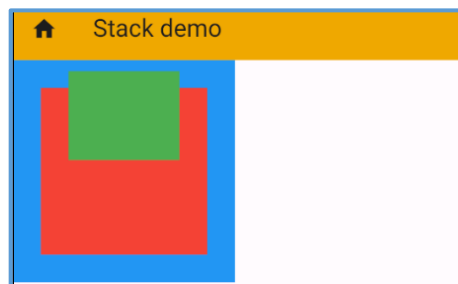
Stack(
  alignment: Alignment.center,
  children: <Widget>[
    Container(
      color: Colors.blue,
      width: 200,
      height: 200,
    ),
    Container(
      color: Colors.red,

```

```

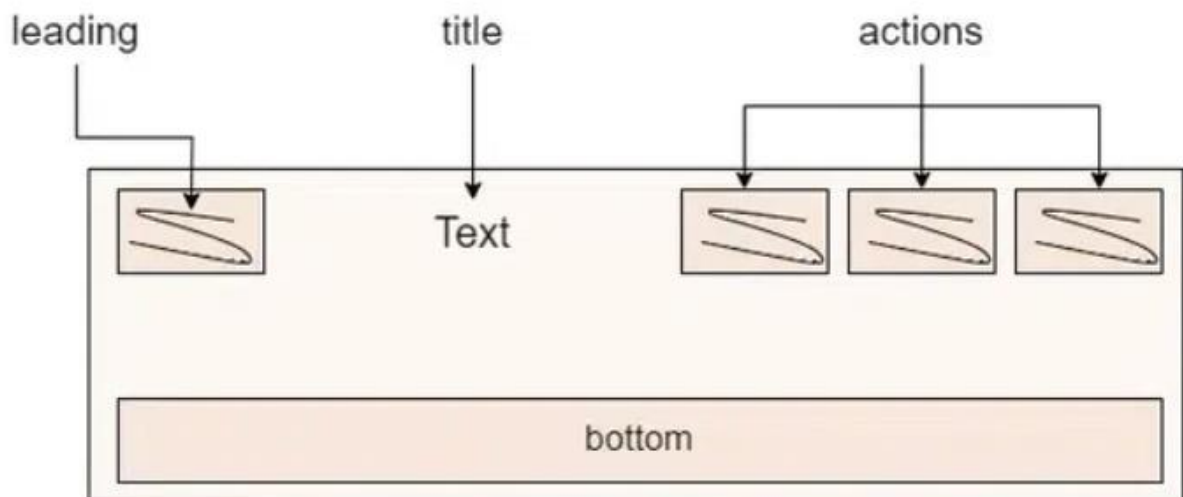
        width: 150,
        height: 150,
      ),
      Positioned(
        right: 50,
        left: 50,
        top: 10,
        child: Container(
          color: Colors.green,
          width: 80,
          height: 80,
        ),
      ),
    ],
  ),
)

```



e. AppBar Widget

- **Đặc điểm** :AppBar widget trong Flutter là một thành phần giao diện người dùng thường được sử dụng để tạo ra thanh tiêu đề (header) cho các trang trong ứng dụng. AppBar cung cấp nhiều thuộc tính và tính năng để tùy chỉnh và quản lý thanh tiêu đề một cách dễ dàng.

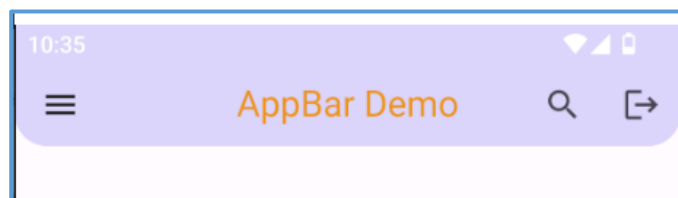


- **Các thuộc tính:**
 - **actions:** Danh sách các biểu tượng (icon) hiển thị trên AppBar .
 - **title:** Tiêu đề hiển thị trên AppBar.
 - **backgroundColor:** Màu nền của Appbar.
 - **elevation:** Vị trí tọa độ z để AppBar.
 - **shape:** hình dạng của AppBar
- **Ví dụ:**

```

appBar: AppBar(
  title: const Text(
    "AppBar Demo",
    style: TextStyle(color: Color.fromARGB(255, 237, 148, 23)),
  ),
  centerTitle: true,
  backgroundColor: Color.fromARGB(41, 31, 4, 239),
  toolbarHeight: 50,
  shape: const RoundedRectangleBorder(
    borderRadius: BorderRadius.only(
      bottomLeft: Radius.circular(20),
      bottomRight: Radius.circular(20)),
  ),
  leading: IconButton(
    icon: const Icon(Icons.menu),
    onPressed: () {},
  ),
  actions: [
    IconButton(
      icon: const Icon(Icons.search),
      onPressed: () {},
    ),
    IconButton(
      icon: const Icon(Icons.logout),
      onPressed: () {},
    ),
  ],
)

```



f. **bottomNavigationBar** widget

- **Đặc điểm**

- Là một widget thuộc Material Design, thường được dùng để hiển thị thanh điều hướng dưới cùng của màn hình.
- Cho phép người dùng chuyển đổi nhanh giữa các màn hình/chức năng chính của ứng dụng (ví dụ: Trang chủ, Cá nhân, Giỏ hàng, v.v.).

- **Các thuộc tính**

- **Items:** Danh sách các **BottomNavigationBarItem**, là các lựa chọn của bottomNavigationBar. Mỗi item gồm
 - icon: Icon hiển thị.
 - label: Nhãn văn bản.
 - backgroundColor: Màu nền của
- **currentIndex:** Xác định được vị trí của item đang được chọn (tính từ 0). Thường kết hợp với hàm cập nhật trạng thái.

- **onTap:** Hàm xử lý khi người dùng bấm vào một item.
- **backgroundColor:** Màu nền cho thanh bottomNavigationBar
- **selectedItemColor & unselectedItemColor:** Màu cho các icon khi chọn và không chọn
- **type:** Xác định cách hiển thị
 - BottomNavigationBarType.fixed (mặc định): Các tab có nền cố định, label luôn hiển thị.
 - BottomNavigationBarType.shifting: Nền và màu sắc của tab thay đổi khi được chọn (phù hợp với 3-5 items).

- **Ví dụ:**

file NavigationBar.dart

```
class BottomNavigationBar_VD extends StatelessWidget {
  const BottomNavigationBar_VD({super.key});
  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      title: "BottomNavigationBar Demo",
      debugShowCheckedModeBanner: false,
      home: MyButtonNavigationBar(),
    );
  }
}
```

file my_bottom_navigation_bar.dart

Xây dựng 2 lớp : MyButtonNavigationBar và MyButtonNavigationBarState.

Do trong class MyButtonNavigationBar kế thừa từ StatefulWidget vì có sự thay đổi về trạng thái khi người dùng thay chọn các item khác nhau trong MyButtonNavigationBar.

```
class MyButtonNavigationBar extends StatefulWidget {
  const MyButtonNavigationBar({super.key});
  @override
  State<MyButtonNavigationBar> createState() => _MyButtonNavigationBarState();
}
```

```
class _MyButtonNavigationBarState extends State<MyButtonNavigationBar> {
  int _selectedIndex = 0;
  static const TextStyle optionStyle =
    TextStyle(fontSize: 30, fontWeight: FontWeight.bold);
  static const List<Widget> _widgetOptions = <Widget>[
    Text(
      'HOME PAGE',
      style: optionStyle,
    ),
    Text(
      'COURSE PAGE',
      style: optionStyle,
    ),
    Text(

```

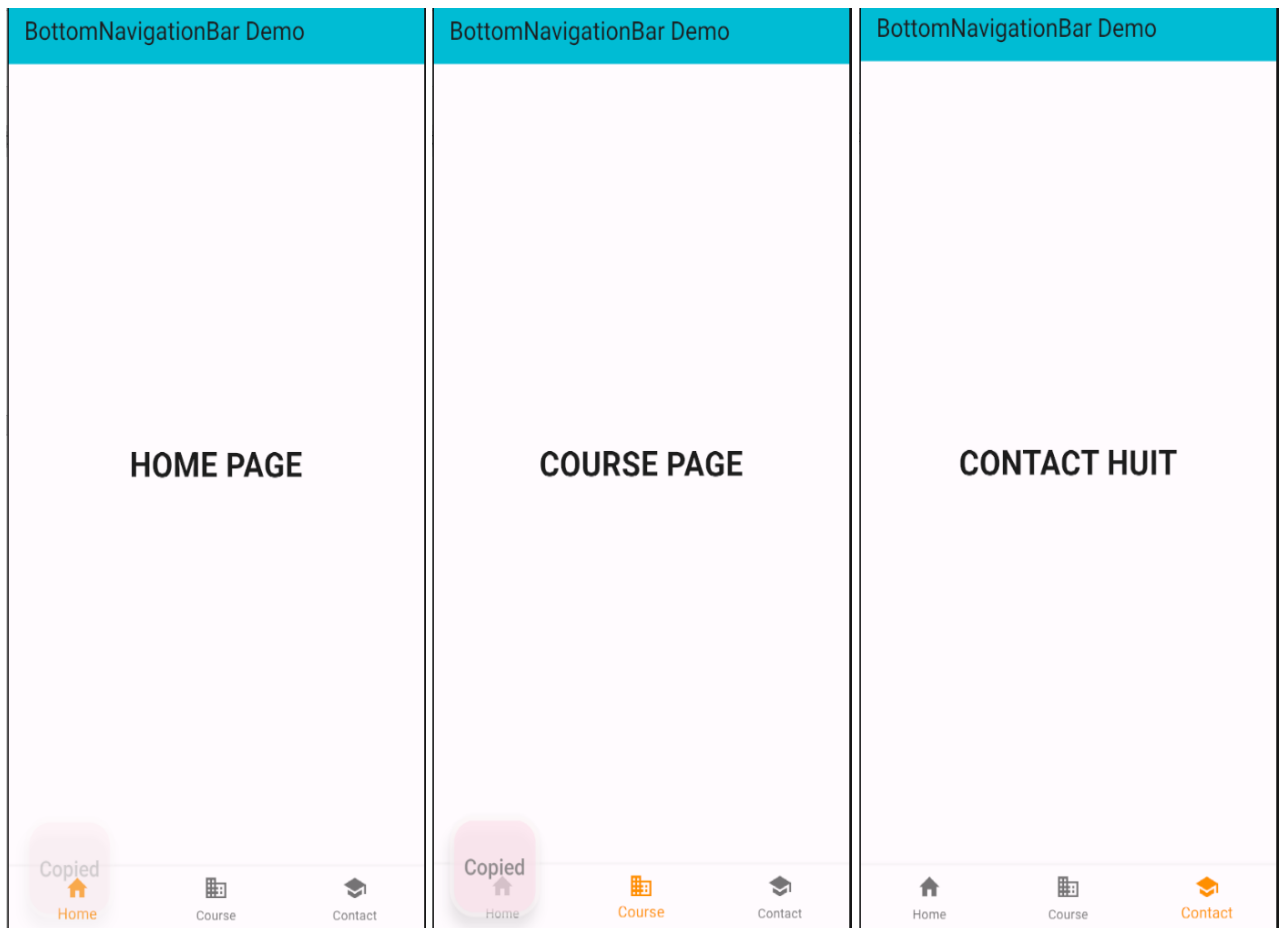
```

        'CONTACT GFG',
        style: optionStyle,
    ),
];

void _onItemTapped(int index) {
    setState(() {
        _selectedIndex = index;
    });
}

@override
Widget build(BuildContext context) {
    return Scaffold(
        appBar: AppBar(
            title: const Text("BottomNavigationBar Demo"),
            backgroundColor: Colors.cyan,
        ),
        body: Center(
            child: _widgetOptions.elementAt(_selectedIndex),
        ),
        bottomNavigationBar: BottomNavigationBar(
            items: const <BottomNavigationBarItem>[
                BottomNavigationBarItem(
                    icon: Icon(Icons.home),
                    label: 'Home',
                    backgroundColor: Colors.cyan,
                ),
                BottomNavigationBarItem(
                    icon: Icon(Icons.business),
                    label: 'Course',
                    backgroundColor: Colors.cyan,
                ),
                BottomNavigationBarItem(
                    icon: Icon(Icons.school),
                    label: 'Contact',
                    backgroundColor: Colors.cyan,
                ),
            ],
            currentIndex: _selectedIndex,
            selectedItemColor: Colors.amber[800],
            onTap: _onItemTapped,
        )),
    }
}

```



g. Drawer widget

- Đặc điểm:

- **Drawer** widget là một thành phần giao diện người dùng cho phép tạo ngăn kéo điều hướng (navigation drawer) thường được hiển thị ở bên trái của màn hình.
- **Drawer** giúp người dùng dễ dàng truy cập vào các mục điều hướng và các tính năng khác của ứng dụng.
- Thường được gắn vào Scaffold.drawer

- Một số thuộc tính

- child: là widget con của drawer.
- hashCode: Mã băm (hash code) cho drawer đang sử dụng.
- elevation: Độ cao của drawer, tạo hiệu ứng đổ bóng.
- semanticLabel: Tiêu đề của Drawer

- Lưu ý

- EndDrawer: Phiên bản mở từ cạnh phải (dùng Scaffold.endDrawer).
- Độ rộng mặc định: Drawer chiếm 80% chiều rộng màn hình, để thay đổi độ rộng drawer có thể cấu hình bằng DrawerTheme
- Cần tổ chức drawer hợp lý, Thêm DrawerHeader, Xử lý sự kiện,...

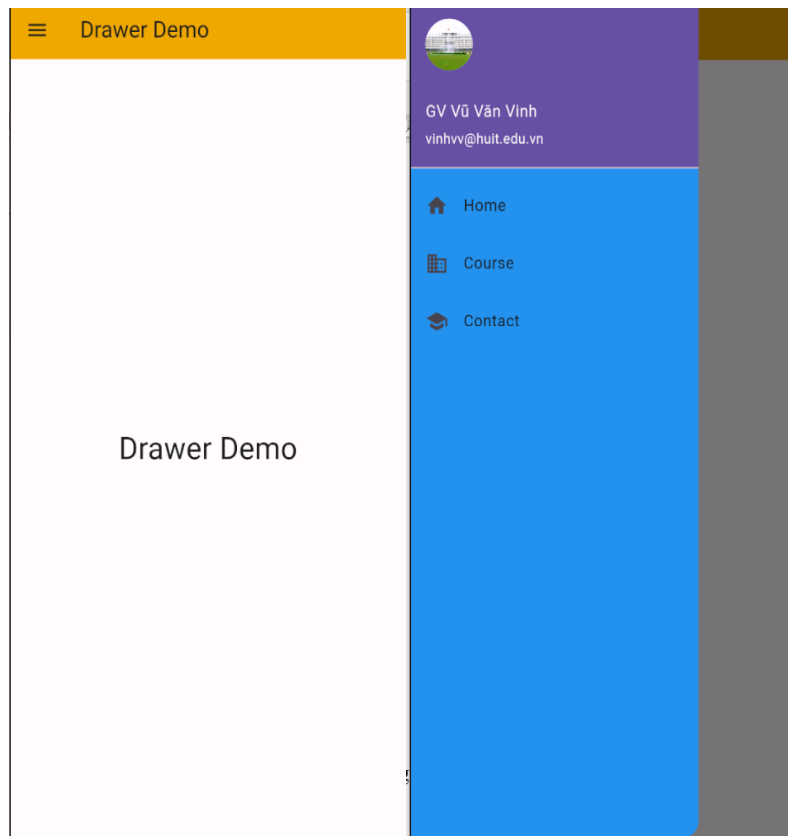
- Ví dụ:

```
class MyDrawerDemo extends StatelessWidget {
  const MyDrawerDemo({super.key});
  @override
```

```

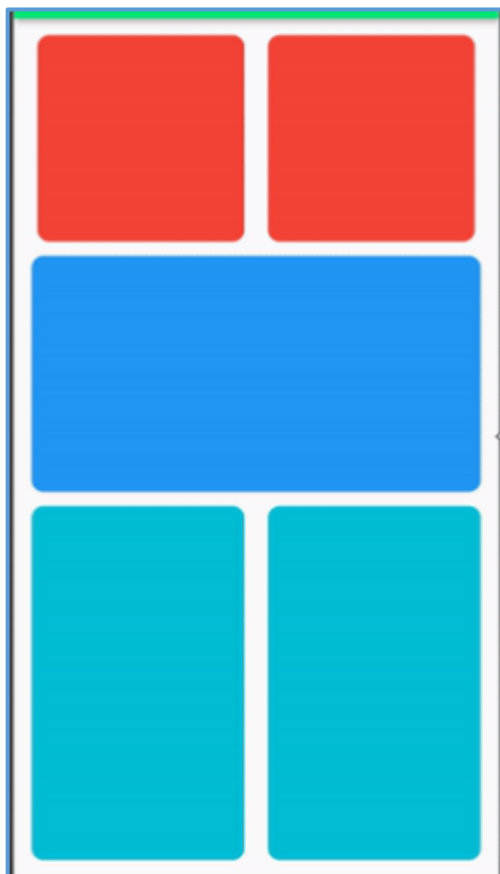
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text("Drawer Demo"),
      backgroundColor: const Color.fromARGB(255, 239, 168, 4),
    ),
    body: const Center(
      child: Text("Drawer Demo", style: TextStyle(fontSize: 30)),
    ),
    drawer: Drawer(
      backgroundColor: Colors.blue,
      child: Column(children: [
        UserAccountsDrawerHeader(
          accountName: Text("GV Vũ Văn Vinh"),
          accountEmail: Text("vinhvv@huit.edu.vn"),
          currentAccountPictureSize: Size.square(50),
          currentAccountPicture: CircleAvatar(
            backgroundColor: Colors.white,
            child: ClipRRect(
              borderRadius: BorderRadius.circular(200),
              child: Image.asset("assets/images/dinh_doc_lap.jpg",
                width: 200, height: 200, fit: BoxFit.cover),
            ),
          ),
        ),
        ListTile(
          leading: const Icon(Icons.home),
          title: const Text("Home"),
          onTap: () {
            Navigator.pop(context);
          },
        ),
        ListTile(
          leading: const Icon(Icons.business),
          title: const Text("Course"),
          onTap: () {
            Navigator.pop(context);
          },
        ),
        ListTile(
          leading: const Icon(Icons.school),
          title: const Text("Contact"),
          onTap: () {
            Navigator.pop(context);
          },
        ),
      ]));
}

```

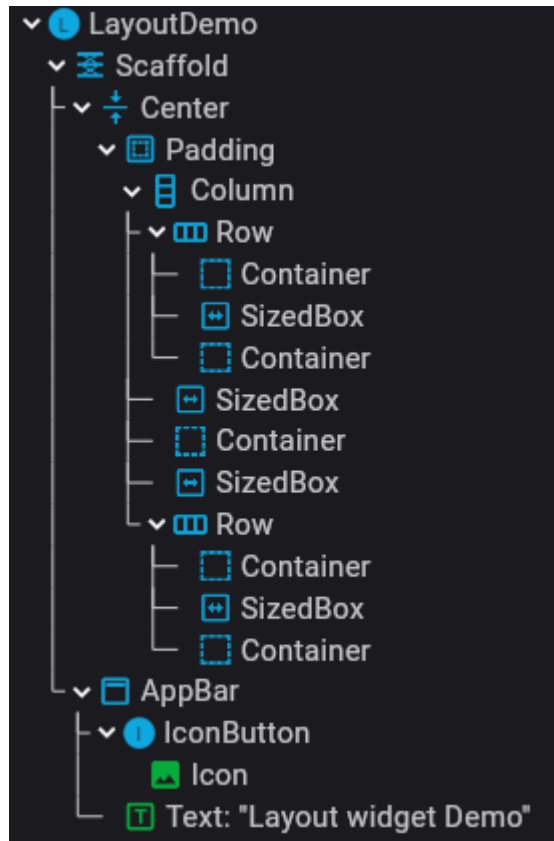


2. Bài tập hướng dẫn

Ví dụ: Xây dựng màn hình hiển thị thông tin của học phần lập trình di động với các nội dung như sau :



Hướng dẫn: Ta sẽ xây dựng cấu trúc của page với widget tree như sau :



file layout_demo.dart

```
Center(
  child: Padding(
    padding: const EdgeInsets.all(14.0),
    child: Column(children: <Widget>[
      Row(
        mainAxisAlignment: MainAxisAlignment.center,
        children: <Widget>[
          Container(
            width: 175,
            height: 175,
            decoration: BoxDecoration(
              borderRadius: BorderRadius.circular(10),
              color: Colors.red,
            ), //BoxDecoration
          ), //Container
          const SizedBox(
            width: 20,
          ), //SizedBox
          Container(
            width: 175,
            height: 175,
            decoration: BoxDecoration(
              borderRadius: BorderRadius.circular(10),
              color: Colors.red,
            ) //BoxDecoration
```

```

        ) //Container
    ],
),
const SizedBox(height: 10),
Container(
    width: 380,
    height: 200,
    decoration: BoxDecoration(
        borderRadius: BorderRadius.circular(10),
        color: Colors.blue),
), //Container
const SizedBox(height: 10),
Row(
    crossAxisAlignment: CrossAxisAlignment.center,
    children: <Widget>[
        Container(
            width: 175,
            height: 300,
            decoration: BoxDecoration(
                borderRadius: BorderRadius.circular(10),
                color: Colors.cyan,
            ), //BoxDecoration
        ), //Container
        const SizedBox(
            width: 20,
        ), //SizedBox
        Container(
            width: 175,
            height: 300,
            decoration: BoxDecoration(
                borderRadius: BorderRadius.circular(10),
                color: Colors.cyan,
            ) //BoxedDecoration
        ) //Container
    ],
),
]),
))

```

Ví dụ: Xây dựng màn hình hiển thị thông tin của học phần lập trình di động với các nội dung như sau :



Hướng dẫn :

- Sử dụng stack widget với 2 widget con

file baitap_stack.dart

```
class BaiTapStack extends StatelessWidget {
  const BaiTapStack({super.key});
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text("Stack demo"),
        backgroundColor: const Color.fromARGB(255, 239, 168, 4),
        leading: IconButton(
          icon: const Icon(Icons.home),
          onPressed: () {},
        ),
      ),
      body: Stack(
        alignment: Alignment.center,
        children: <Widget>[
          Container(
            decoration: BoxDecoration(
              color: Colors.blue,
              borderRadius: BorderRadius.circular(40),
              image: const DecorationImage(
                image: AssetImage('assets/images/dong_phong_nha.jpg'),
                fit: BoxFit.cover,
              ),
            ),
          ),
        ],
      ),
    );
  }
}
```

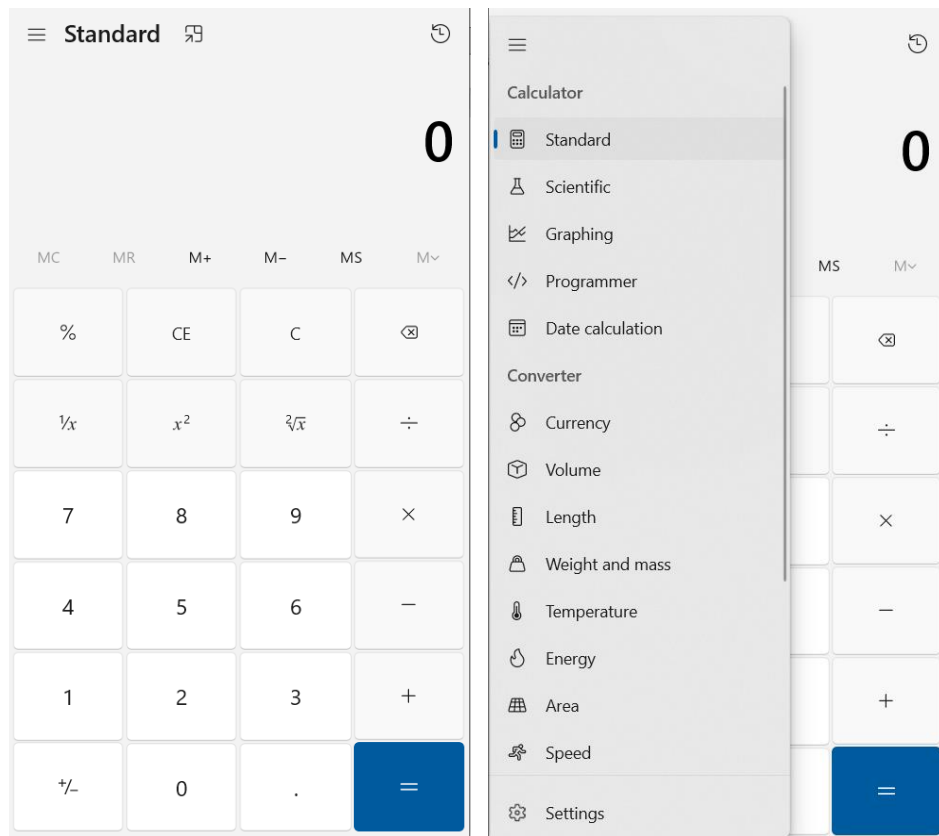
```

    ),
  ),
  Positioned(
    right: 20,
    left: 20,
    bottom: 20,
    child: Container(
      decoration: BoxDecoration(
        color: Colors.white,
        borderRadius: BorderRadius.circular(10),
      ),
      height: 150,
      child: const Column(
        children: <Widget>[
          SizedBox(height: 10),
          Text(
            "Động Phong Nha",
            style: TextStyle(
              fontSize: 24,
              color: Colors.red,
              fontWeight: FontWeight.bold,
            ),
          ),
        ],
      ),
      Padding(
        padding: EdgeInsets.all(8.0),
        child: Text(
          "Động Phong Nha, nằm trong vườn quốc gia Phong Nha - Kẻ
          Bàng, tỉnh Quảng Bình, Việt Nam, là một trong những hang động nổi tiếng và hấp
          dẫn nhất trên thế giới.",
          maxLines: 4,
          overflow: TextOverflow.clip,
          style: TextStyle(
            fontSize: 15,
            fontWeight: FontWeight.normal,
          ),
        ),
      ),
    ],
  ),
),
],
));
}
}

```

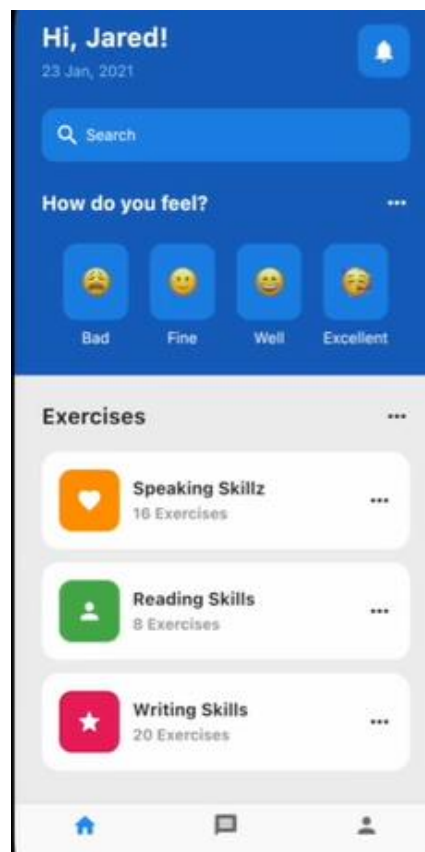
3. Bài tập tại lớp :

Bài tập 01. Hãy thiết kế ứng dụng máy tính trên thiết bị di động



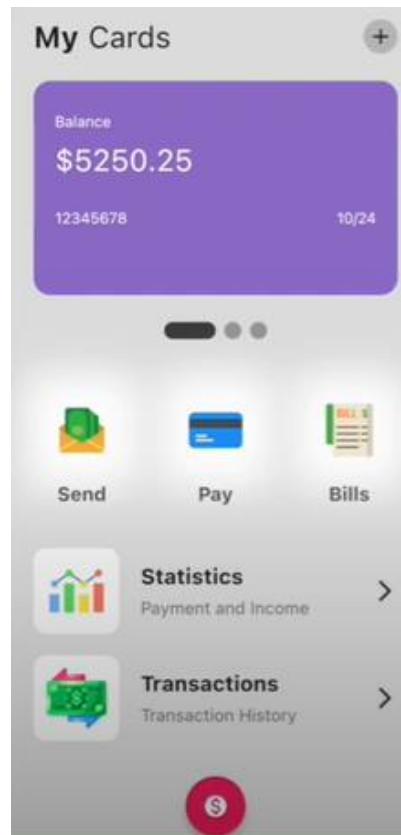
Bài tập 02. Sử dụng `bottomNavigationBar` xây dựng ứng dụng giới thiệu cơ sở vật chất của trường Đại học Công thương thành phố hồ Chí Minh như sau :

Bài tập 03. Thiết kế giao diện cho phép người dùng thể hiện tình trạng của một người dùng trong ngày

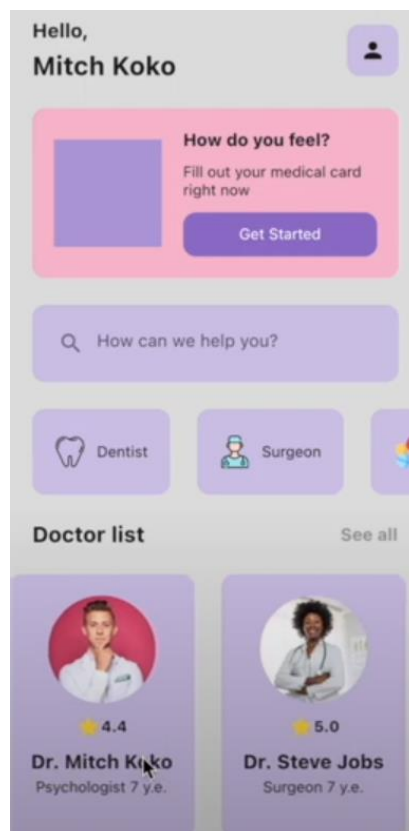


4. Bài tập về nhà :

Bài tập 04. Thiết kế giao diện mô phỏng chức năng của ví điện tử của khách hàng với giao diện như sau



Bài tập 05. Thiết kế giao diện đăng ký khám chữa bệnh online như sau



Bài tập 06. Xây dựng ứng dụng màn hình nghe nhạc như sau :

